Advanced AI Medical Intelligence Platform

Deep Learning · Explainable AI · LLM-Assisted Reporting

Chest X-Ray Pneumonia Detection System

Project Report

Submitted for:	AI/ML Engineer (Fresher) — Technical Assignment
Organization:	SN Matrix Software Pvt. Ltd.
Candidate Name:	[Your Name Here]
Submission Date:	[Fill in submission date]
GitHub Repository:	[Paste your GitHub repo URL here]
Live Deployment URL:	[Paste your deployed app URL here, if applicable]

1. Project Objective

The objective of this project is to design and implement a complete, end-to-end AI-powered medical intelligence platform capable of analyzing chest X-ray images, predicting pneumonia using a deep learning model, explaining the model's predictions using Grad-CAM (an Explainable AI / XAI technique), automatically generating a human-readable draft medical report using a Large Language Model (Anthropic Claude), exposing all functionality through a REST API, persisting prediction history in a database, and providing a simple, user-friendly web interface. The system is also containerized with Docker for portable deployment.

Disclaimer: This system is built strictly for educational / technical-assessment purposes. It is not a certified medical device, has not undergone clinical validation, and must never be used as a substitute for professional medical diagnosis.

2. Problem Statement & Dataset

Task: Binary image classification — determine whether a chest X-ray shows a **NORMAL** lung or a lung with signs of **PNEUMONIA**.

Dataset: Chest X-Ray Images (Pneumonia), originally published by Kermany et al. (*Cell*, 2018), distributed publicly via Kaggle (kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia). It contains 5,863 JPEG chest X-ray images split into train / validation / test sets, each divided into NORMAL and PNEUMONIA classes. The dataset exhibits class imbalance (roughly 3x more PNEUMONIA images than NORMAL images in the training split), which was explicitly addressed in the modeling approach (see Section 4).

Note on dataset access: the dataset was not bundled inside the submitted repository due to its size (~1.2GB) and Kaggle's redistribution terms. Instructions for downloading it are provided in `data/README.md` of the repository.

3. System Architecture

The system follows a modular pipeline design, separating concerns cleanly between the deep-learning model, the explainability layer, the language-model reporting layer, and the serving/storage layer:

```
Chest X-Ray Image → Preprocessing (resize / normalize) → DenseNet121 CNN (prediction) →
Grad-CAM (visual explanation) → Claude LLM (structured-data-to-text report) → FastAPI
REST layer → SQLite/PostgreSQL database (history) → Web UI (display).
```

A key design decision is that the LLM is **never given the raw image** — it only receives already-computed structured numeric output from the deep learning model (predicted class, confidence score, full probability breakdown, and a short textual description of the Grad-CAM attention region). This guarantees the LLM cannot hallucinate visual findings it never actually observed, and keeps a clean separation between 'what was found' (the CNN's job) and 'how it is phrased' (the LLM's job).

4. Deep Learning Model

4.1 Architecture

Backbone: DenseNet121, pretrained on ImageNet, fine-tuned end-to-end (or with the convolutional backbone optionally frozen for a faster first pass). DenseNet121 was chosen because it is the most widely validated CNN backbone for chest X-ray classification in the medical-imaging literature (notably in Stanford's CheXNet project), and its densely-connected layers provide strong gradient flow which in turn produces cleaner Grad-CAM localization maps.

The original 1000-class ImageNet classification head was replaced with a custom head: Linear(1024→256) → ReLU → Dropout(0.3) → Linear(256→2).

4.2 Training Methodology

- Transfer learning: leveraging ImageNet-pretrained weights rather than training from scratch, since the dataset (~5k images) is too small to train a deep CNN from random initialization.
- Class-weighted Cross-Entropy Loss to counter the NORMAL/PNEUMONIA class imbalance.
- Data augmentation on the training set only: random horizontal flip, small random rotation (±7°), and color jitter — chosen conservatively so as not to distort diagnostically relevant anatomical structure.
- Adam optimizer with a ReduceLROnPlateau learning-rate scheduler (halves LR after 2 epochs without validation-accuracy improvement).
- Best checkpoint selection by validation accuracy, with final reported metrics computed once on the held-out test set only (never used for any training decision).

4.3 Results

[FILL IN AFTER TRAINING — run `python -m src.train` then `python -m src.evaluate`, and paste your numbers from `reports/test_classification_report.json` below. Typical results reported in prior work on this exact dataset/architecture are in the 90-95% test-accuracy range with 0.95+ recall on the PNEUMONIA class.]

Metric	Value
Test Accuracy	[fill in]
Precision (PNEUMONIA)	[fill in]
Recall (PNEUMONIA)	[fill in]
F1-score (PNEUMONIA)	[fill in]
ROC-AUC	[fill in]

[Insert your confusion matrix image here: reports/confusion_matrix.png]
[Insert your ROC curve image here: reports/roc_curve.png]

5. Explainable AI (Grad-CAM)

Grad-CAM (Gradient-weighted Class Activation Mapping, Selvaraju et al., ICCV 2017) was implemented from scratch (not via an external library) to keep the mechanism fully transparent. The method works by: (1) capturing the activations of the last convolutional layer via a forward hook, (2) capturing the gradient of the predicted class score with respect to those same activations via a backward hook, (3) global-average-pooling the gradients to obtain a per-channel importance weight,

(4) computing a weighted sum of the activation maps followed by a ReLU, producing a coarse localization heatmap, which is then upsampled and overlaid on the original X-ray. This heatmap highlights the lung regions the model 'looked at' most strongly when making its prediction, which is essential for clinical trust and for detecting cases where the model may be relying on spurious correlations rather than genuine pathology.

[Insert 2-3 example Grad-CAM overlay images here from reports/gradcam_outputs/, one for a correctly predicted PNEUMONIA case and one for a correctly predicted NORMAL case.]

6. LLM Integration

Anthropic's Claude API (model: claude-sonnet-4-6) is used to convert the structured output of the deep learning model into a clearly-written, professionally-formatted draft report with four sections: Findings, Impression, Recommendation, and Disclaimer. The system prompt explicitly instructs the model to (a) never assert a diagnosis with certainty, always hedging with language like 'the model predicts' / 'suggests', (b) always include a bolded disclaimer stating the report is AI-generated, not a diagnosis, and requires physician review, and (c) rely only on the structured data provided, not on any independent visual analysis. If no API key is configured, the system gracefully falls back to a deterministic, rule-based templated report, so the full application remains demoable without any external dependency — a resilience pattern worth highlighting for a production system.

7. API Development

A FastAPI application (api/main.py) exposes the full functionality as a REST API with auto-generated OpenAPI/Swagger documentation at /docs.

Method	Endpoint	Description
GET	/api/health	Service and model-loaded status
POST	/api/predict	Upload an X-ray image, get prediction + Grad-CAM + LLM report
GET	/api/history	Paginated list of past predictions
GET	/api/history/{id}	Full detail of a single past prediction
DELETE	/api/history/{id}	Delete a prediction record
GET	/gradcam/{filename}	Serves a saved Grad-CAM overlay image

Input validation includes content-type checking (JPEG/PNG only), a file size limit (10MB), and a clear 503 error if no trained model checkpoint is available yet. CORS middleware is enabled for frontend/backend decoupling.

8. Database Design

Prediction history is persisted using SQLAlchemy ORM, defaulting to SQLite for zero-config local/demo use, and swappable to PostgreSQL in production purely via the DATABASE_URL environment variable — no code changes required. The `predictions` table schema:

Column	Type	Description
id	Integer (PK)	Auto-incrementing primary key

original_filename	String	Uploaded file's original name
predicted_class	String	NORMAL or PNEUMONIA
confidence	Float	Model confidence for the predicted class
probability_normal / probability_pneumonia	Float	Full softmax probability breakdown
attention_region	String	Grad-CAM textual attention location
gradcam_image_path	String	Filename of the saved heatmap overlay
llm_report	Text	Full LLM-generated (or fallback) report text
created_at	DateTime	Timestamp of the prediction

9. Web Application

A lightweight single-page frontend (plain HTML/CSS/JavaScript, no build step required) is served directly by the FastAPI app. It provides drag-and-drop image upload, live display of the original image side-by-side with its Grad-CAM overlay, a confidence bar and probability breakdown, the generated LLM report, and a history tab listing all past predictions.

[Insert 1-2 screenshots of the running web application here.]

10. Deployment

The application is fully containerized via a Dockerfile (python:3.11-slim base image) and a docker-compose.yml for one-command startup with volume-mounted model weights and database. For a public-facing deployment, the Dockerfile can be deployed as-is to any container hosting platform (Render, Railway, Fly.io, Hugging Face Spaces).

[FILL IN: Live Deployment URL, if you deployed the app publicly]
[FILL IN: which platform you used and any deployment notes]

11. Testing & Quality Assurance

A pytest suite (tests/test_pipeline.py) verifies the model's forward pass produces correctly shaped output, Grad-CAM produces a valid [0,1]-normalized heatmap, the image denormalization and overlay functions produce correctly shaped/typed arrays, the attention-region text generator produces sensible output, image transforms produce correctly shaped tensors, and the LLM fallback report generator always includes the required disclaimer. These tests use small synthetic images so they can run in seconds without requiring the real dataset or a GPU — useful for quick CI-style verification before committing to a full training run.

During development, this test suite caught a real bug: a missing `.detach()` call before a `.numpy()` conversion in gradcam.py, which would have crashed in a Grad-CAM code path. The fix is included in the submitted code.

12. Ethical & Safety Considerations

- The system is explicitly framed as a decision-support / draft-report tool, never as an autonomous diagnostic authority.
- The LLM is deliberately never given the raw image — only pre-computed structured numeric output — to prevent it from hallucinating visual findings.
- Every generated report includes a clear, bolded disclaimer stating it is AI-generated, is not a diagnosis, and requires review by a licensed physician.
- Class-weighted training mitigates a common failure mode in medical-imaging ML: a model that achieves high accuracy simply by favoring the majority class.
- Grad-CAM visualizations allow a human reviewer to sanity-check whether the model is attending to clinically plausible regions rather than spurious image artifacts.

13. Conclusion

This project implements a complete, modular, end-to-end AI medical intelligence pipeline covering every stage requested in the assignment brief: a deep-learning diagnostic model, Explainable AI via Grad-CAM, LLM-assisted natural-language reporting, a documented REST API, persistent database storage, a usable web interface, and container-based deployment. [Add 2-3 sentences here reflecting on your specific experience training the model, any challenges you faced, and what you would improve with more time/compute.]

14. References

- Kermany, D. S., et al. "Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning." Cell, vol. 172, no. 5, 2018, pp. 1122-1131.
- Selvaraju, R. R., et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization." ICCV, 2017.
- Rajpurkar, P., et al. "CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning." arXiv:1711.05225, 2017.
- Huang, G., et al. "Densely Connected Convolutional Networks." CVPR, 2017.
- Kaggle Dataset: Chest X-Ray Images (Pneumonia) — kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia